Unit-1

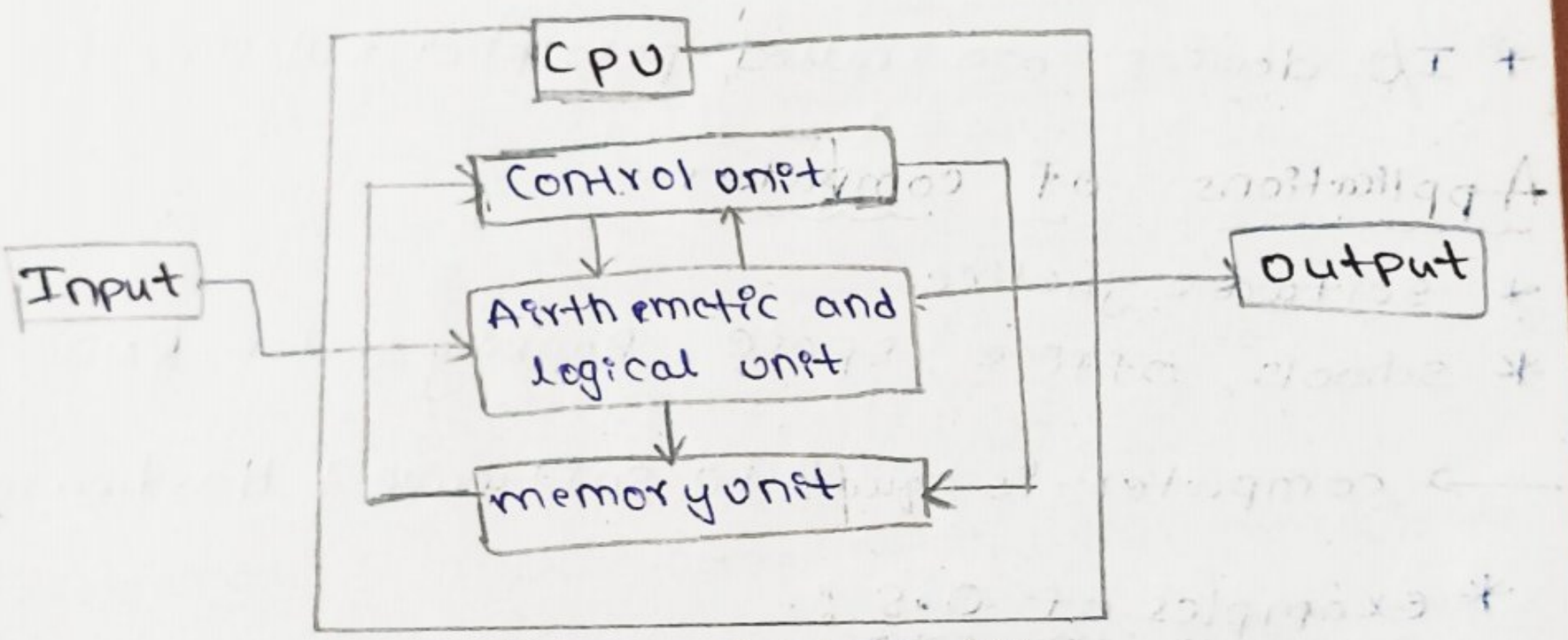
Basic structure of Computers

computer :- commonly operating mechine purposely used for technical and educational research.

→ Two types of computer

1. Analog computer 2. Digital computer

functional units / functional Blocks / Block diagram of computer



Block diagram of a computer

Art of Testing

Input unit :- used to give input data using input devices

Ex :- Keyboard, mouse, scanner, etc.

CPU :- Heart of the system

→ It contains processor or controller.

→ control unit :- used to control sequence of operation

→ ALU :- used to perform Airthematic logical & shift operations

Ex :- +, −, ×, /, Increment, Decrement → Airthematic

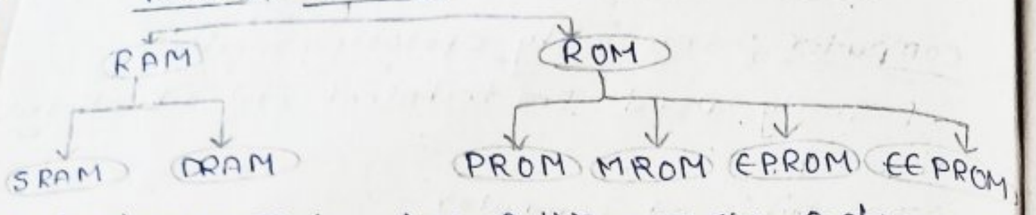
AND, OR, NOT → logical

→ shift operations :- used to shift the data

Ex :- >> (SHR), << (SHL), ASHR, ASHL, ROR, ROL

→ Memory :- used to store the intermediate / final results.

Primary Memory



* Auxilary memory, Associative memory, cache memory, virtual memory are the types of memorys.

Output units :- Used for displaying the results

Ex :- Monitor, printer, speaker, Headset, etc.

* I/O devices are called perepipheral devices

Applications of computers

* Software justice
* schools, office, space, Banking sector, RND.

→ computer is equal to software + Hardware

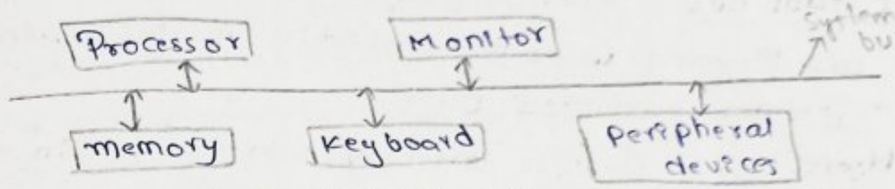
* examples of O.S :-

→ windows, Microsoft, LINUX, UNIUX, MAC, Compiler, Interpreter.

Operation of computers :-

1. To give input
2. Operating on Data
3. store the data.
4. Display the output

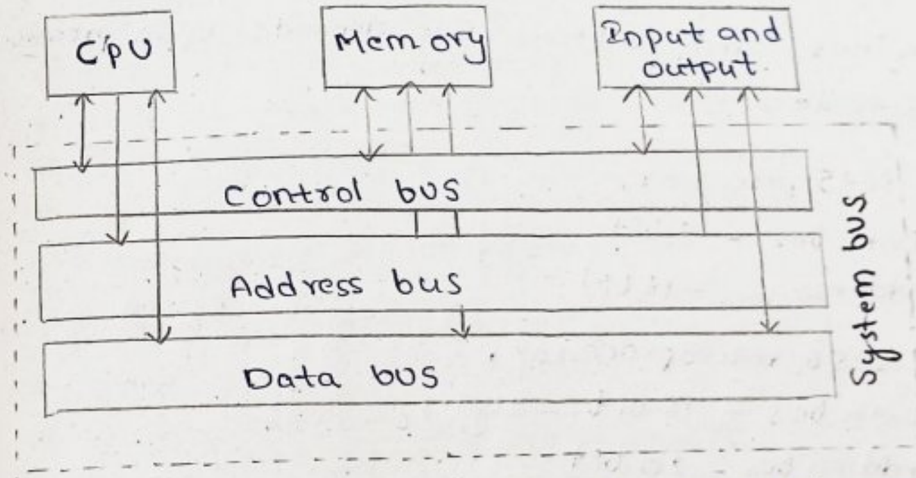
Basic structure of computer :-



System bus :- It is electronic cables connected to codes of each and every device

→ It is devided into 3 parts.

1. Address BUS 2. Data BUS 3. control BUS



System bus :- " Single computer bus that connects the major components of computer system combining the functions of data bus to carry data, Address Bus to determine where it should be sent or Recieve from, Control bus for controlling sequence of operations".

* System bus is used for,
  1. carring information from one device to other
  2. supplying the power to the peripherals.
  3. providing the clk signal for synchronization.

functions of system bus

1, 2, 3 are called

1. control Bus :- Carries control signals for controlling and coordinating various activities in this system
→ It is under controlled Unit.

2. Address Bus :- Carries Memory address within the system. (16 bit)

3. Data Bus :- (8 bit) Transfer of data from one location to other within the system.
→ Data can be Image, audio, video, Text.

* Data Bus and address Bus depends upon processor or controllers.

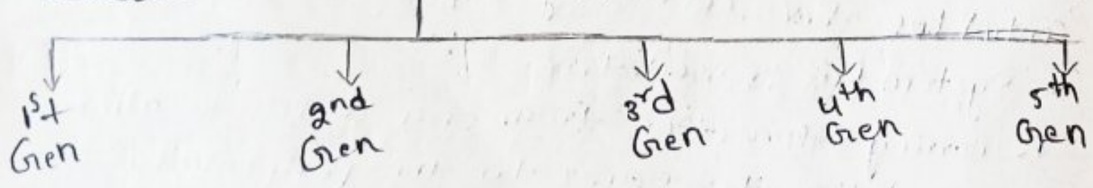
* for 8085 processor,
  Data bus - 8 bit — 1 byte
  Address bus - 16 bit.

* for 8086 microprocessor,
  Data bus - 16 bit — 2 byte
  Address bus - 20 bit

(processors)

* for 8051 controller,
  Data bus - 8 bit — 1 byte
  Address bus - 16 bit

Generations of Computer.



1st Generation :- (1940 - 1956)
* Technology used vaccum tubes
* Magnetic drums used for storing data
* power consumption is very high.
* size complex
* Machine level language is used
* No standard architecture.
* Very expensive
* Reliability less. * Non portable.

2nd Generation :- (1956 - 1963)
* Technology used Transistor (Shortley, Brattain, Bardeen)
* compare to 1st faster
* power req large.
* Assembly language used.
* size less compares to 1st Gen.
* Magnetic tapes and discs are used for storing data
* Reliability high.
* complex programs can be written.

3rd Generation :- (1964 - 1971)
* Technology used ICS (Jale, kibey) — Integrated circuit
* Small size
* faster execution.
* energy efficiency
* Improved Reliability
* high level languages are used cobalt, fortron.
* Multiprogramming is possible.

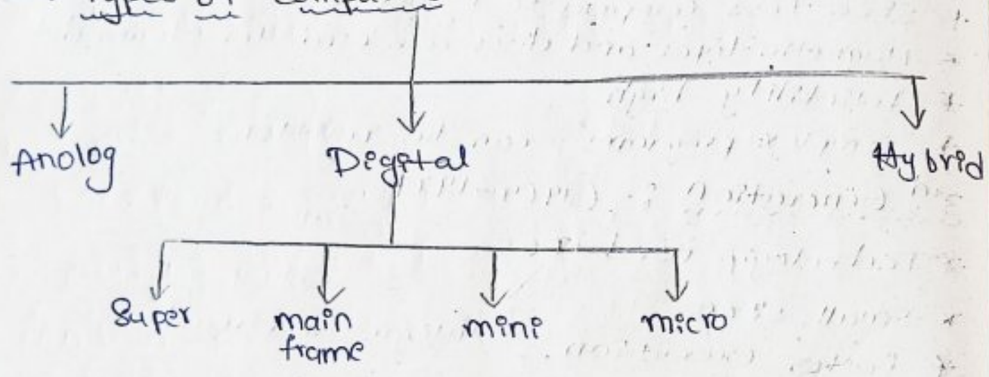
4th Generation :- (1971 - 1985)
* Technology used VLSI / ULSI
* VLSI → very large scale Integrated circuit
* ULSI → ultra large scale Integrated circuit
* Decrease in size and cost
* Improved power efficiency
* Increase in Reliability
* High speed communication
* Usage of ~~Lane~~ LAN → local area network

LAN, WAN, MAN

5th Generation :- (1985 - present)

* Technology used AI
* used for ~~double~~ development & Research
* Audio, video compressing techniques
* high speed execution, parallel processing
* Multi pipelining pipe-lining
* power req less & size also less
* portable.

→ Types of computers.



1. Analog computers :-

* These are used for displaying the continuous graph
* It is still used in mechanical, electrical & optical domains
* These are replaced by digital computers.

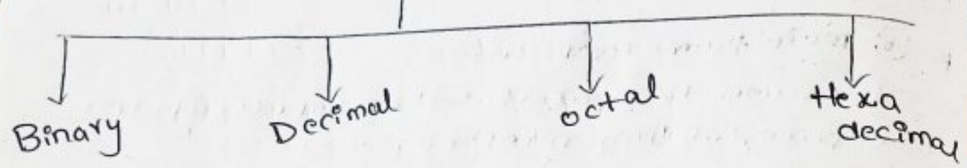
2. Digital computers :-

* Stores the data 0's and 1's.
* ⓐ super computers :-
  → high powerful computers
  → solves comple computational problems
  → cost, performance, ~~high~~.
  → time low.
  → used in space investigation, Atomic weapons, genetic engineering, Military applications, wheather forcasting.
* ⓑ main frame computers :-
  → these are less powerful than supercomputers, more powerful than mini computers
  → execution time in b/w super & mini
  → cost high.
  → used in. Banking sectors, Railways, Airlines, stock exchange, E-Business.
* ⓒ mini computers :-
  → performance is b/w mainframe & micro
  → used for personal use.
  → ex :- PDP-8, tabs etc--.
  → power consumption is low.
  → No need of maintainence.
* ⓓ micro computers :-
  → Small size, low cost, portable, low power consumption.
  → less speed.
  → ex :- mobile, laptop, deskops etc.

3. Hybrid computer :-

→ It can acts as Analog & digital
→ ADC converts are used for converting Analog to digital
→ DAC converts are used for converting digital to Analog.

Number systems :-



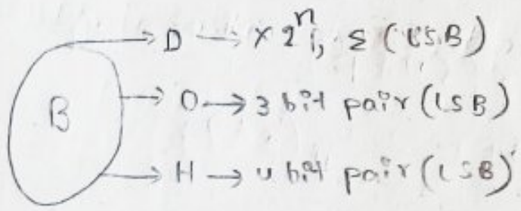
Explination :-

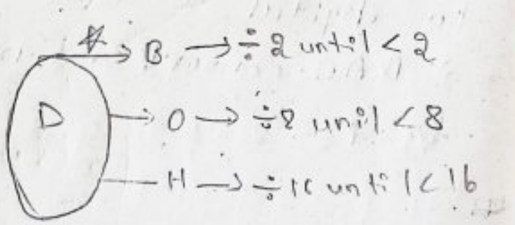
1. **Binary :-**
* contains 0's & 1's
* also called as Machine language
* Base / radix is 2

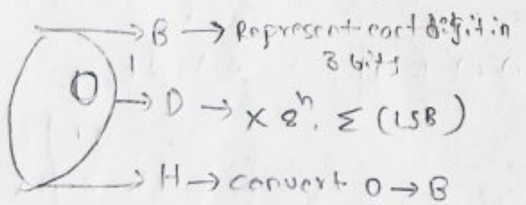
2. **Decimal :-**
* contain 0 - 9
* Base / radix is 10

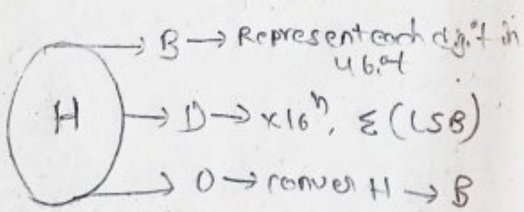
3. **Octal :-**
* contain 0 - 7
* Base / radix is 8.

4. **Hexa decimal :-**
* contain 0-9, A → f
* Base / radix is 16









① convert $(32)_{10}$ to remaining three

Sol $(32)_{10} \rightarrow (?)_2$ $(32)_{10} \rightarrow (\ )_8$

2 | 32
2 | 16 — 0
2 | 8 — 0
2 | 4 — 0
2 | 2 — 0
1 — 0

8 | 32
4 — 0

$(32)_{10} = (40)_8$

16 | 32
2 — 0

$(32)_{10} = (20)_{16}$

$(32)_{10} = (100000)_2$

② convert $(111000)_2$ to remaining three.

Sol $(111000)_2 \rightarrow (?)_{10}$.

$= 0\times2^0 + 0\times2^1 + 0\times2^2 + 1\times2^3 + 1\times2^4 + 1\times2^5$

$= 0 + 0 + 0 + 8 + 16 + 32$

$= 56$

$(111000)_2 = (56)_{10}$

$(111\ 000)_2 \rightarrow (?)_8$.

7 0

$\Rightarrow (111000)_2 = (70)_8$

$(0\,0\,111000) \rightarrow (?)_{16}$

$\Rightarrow (111000)_2 = (38)_{16}$

③ convert $(77)_8$ to remaining three

Sol $(77)_8 \rightarrow (?)_2$

111 111

$(77)_8 = (111111)_2$

$(77)_8 \rightarrow (?)_{10}$.

$= 7\times8^0 + 7\times8^1 = 7 + 56 = 63$

$\Rightarrow (77)_8 \rightarrow (63)_{10}$

Scanned with OKEN Scanner

$(77)_8 \longrightarrow (\ ?\ )_{16}$

111 111

$\rightarrow$ 001111111

$\Rightarrow \boxed{(77)_8 \longrightarrow (3f)_{16}}$

④ convert, $(fA2)_{16}$ into remaining 3.

sol $(fA2)_{16} = (\ ?\ )_2$

1111 1010 0010

$\Rightarrow \boxed{(fA2)_{16} = (1111\ 1010\ 0010)_2}$

$(fA2)_{16} = (\ ?\ )_{10}$

$= 2\times16^0 + A\times16^1 + f\times16^2$

$= 2\times1 + 10\times16 + 15\times256$

$= 4102$

$\boxed{(fA2)_{16} = (4102)_{10}}$

$(fA2)_{16} = (\ ?\ )_8$

1111 1010 0010

$\Rightarrow \underline{111}\ \underline{110}\ \underline{100}\ \underline{010}$

7 6 4 2

$\Rightarrow (7642)_8$

$\Rightarrow \boxed{(fA2)_{16} = (7642)_8}$

⑤ Convert $(10.05)_{10}$ into Binary.

sol $10.05 = 10 + 0.05$

2 | 10

2 | 5 — 0

2 | 2 — 1

1 — 0

$\rightarrow (10)_{10} = (1010)_2$

$0.05 \times 2 = 0.10 = 0$

$(1010.0)_2$

$\therefore \boxed{(10.05)_{10} = (1010.0)_2}$

⑥ convert 10.3 into Binary.

sol $10.3 = 10 + 0.3$.

$10 \longrightarrow (1010)_2$.

$0.3 \times 2 = 0.6 = 0$

$0.6 \times 2 = 1.2 = 1$

$0.2 \times 2 = 0.4 = 0$

$0.4 \times 2 = 0.8 = 0$

$0.8 \times 2 = 1.6 = 1$

$\therefore \boxed{(10.3)_{10} = (1010.01001)_2}$

Complements $\longrightarrow (r, r-1)$

1's 2'sc | 9's, 10'sc | 7's, 8'sc | 15's, 16'sc

① Calculate $(r, r-1)$'s c of $(7)_{10}$

sol 9's c $\rightarrow$ 9 − 7 = 2 ; 10'sc $\rightarrow$ 2 + 1 = 3

② Calculate (7, 8'sc) of $(77)_8$

sol 7'sc $\rightarrow$ 77 − 77 = 00 ; 8'sc $\rightarrow$ 00 + 1 = 01

③ Calculate 15, 16'sc of $(f6)_{16}$

sol 15'sc $\rightarrow$ ff − f6 = 09 ; 16'sc $\rightarrow$ 09 + 1 = A

Scanned with OKEN Scanner

** Calculate 1's & 2's c of $(17)_{10}$.

Sol

2 | 17
2 | 8 — 1
2 | 4 — 0
2 | 2 — 0
1 — 0

$(10001)_2$

1's c → 01110

2's c → 01110 + 1 = 01111

(5) find the value of −43. in Binary.

2 | 43
2 | 21 — 1
2 | 10 — 1
2 | 5 — 0
2 | 2 — 1
1 — 0

→ $(43)_{10}$ → $(101011)_2$

1's c → 010100

2's c → 010100 + 1 = 010101

∴ $(-43) = (010101)_2$

(6) Calculate 7's & 8's c of $(111001)_2$

$(111001)_2 \rightarrow (71)_8$

Sol

7's c → 77 − 71 = 6

8's c → 6 + 1 = 7

$(111001) = (67)_8$

classification of Binary codes

- Weighted codes (or) Positional codes. Ex :- 8421, 2421
- Non weighted codes (or) non positional. Ex :- excess-3, grey code, excess 3 grey, Hamming code, cyclic code
- alphanumeric codes. Ex :- ASCII

Decimal	8421	2421	excess-3	grey	Excess 3 grey
0	0000	0000	0011	0000	0010
1	0001	0001	0100	0001	0110
2	0010	0010	0101	0011	0111
3	0011	0011	0110	0010	0101
4	0100	0100	0111	0110	0100
5	0101	1011	1000	0111	1100
6	0110	1100	1001	0101	1101
7	0111	1101	1010	0100	1111
8	1000	1110	1011	1100	1110
9	1001	1111	1100	1101	1010

(1) convert 24 → 8421, excess 3, grey, excess 3 grey

Sol

2 | 24
2 | 12 — 0
2 | 6 — 0
2 | 3 — 0
1 — 1

→ 11000.

8421 → not possible

Excess 3 → 24 + 33 = 57 → 01010111

grey → 11000. → 10100

excess grey code

0101011

↓

0111100

24
+ 33
57

2 | 57
2 | 28 - 1
2 | 14 - 0
2 | 7 - 0
2 | 3 - 1
1 - 1

111001

↓

1001101

(2) Convert $(1111)_2$ from grey to binary.

Sol

1111

↓

1010

* Alphanumeric codes :-

American standard code for information interchange.

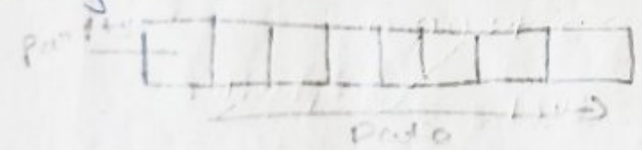
→ It is a 8 bit code out of which 7 bits → data

8th bit → parity

Parity is used for error checking and error correctin

Parity can be even or odd.

Parity bit is added as MSB.



* It contains 128 characters out of which

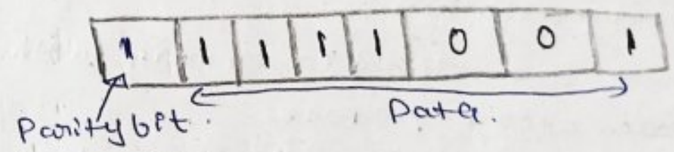
1. 95 Uppercase, lowercase letters, 0-9 numbers
2. 23 format functional keys 10 - for data communication.

Decimal	Binary x y z	odd parity	even parity
0	0 0 0	1	0
1	0 0 1	0	1
2	0 1 0	0	1
3	0 1 1	1	0
4	1 0 0	0	1
5	1 0 1	1	0
6	1 1 0	1	0
7	1 1 1	0	1

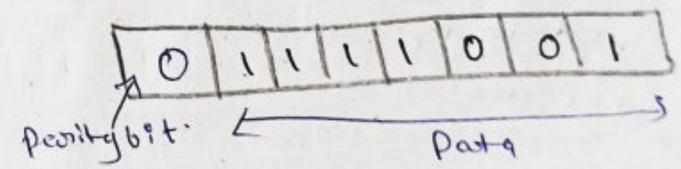
Example :-

find the received ~~data~~ if transmited data is 1111001 in even, parity and odd parity.

(i) even parity.



(ii) odd parity.



* Odd parity circuit :-

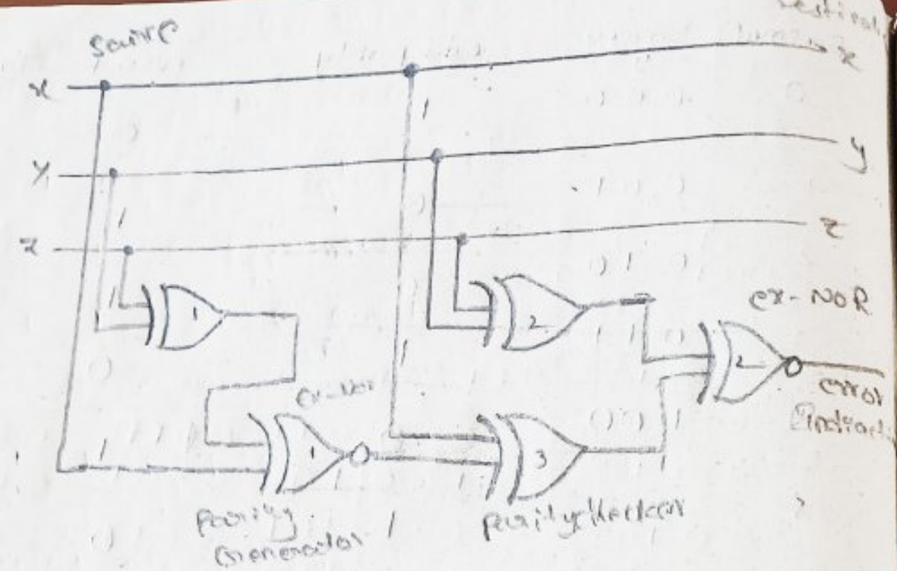
→ It contains parity generators.

→ parity checker.

→ error indicator.

→ constructed with ex-OR, ex-NOR gates.

Scanned with OKEN Scanner



Case-1 :- x=0; y=0; z=0
Parity Generator = 1
Parity checker = same '1'
error indicator = 0

Case-2 :- x=1; y=1; z=1
Parity generator = 0
Parity checker = 1
error indicator = 0

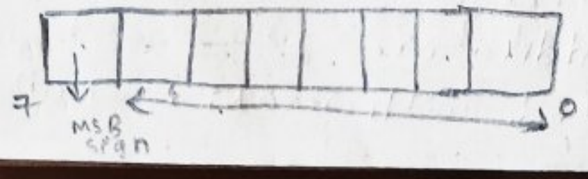
fixed point Representation :-

→ Real numbers are stored in memory with standard formats. for calculation purpose.

→ fixed point representation has 3 parts,
1. signed magnitude representation
2. 1's comp Representation
3. 2's comp Representation

→ we can Represent both (+) & (-) signed (or) unsigned number.

format :-

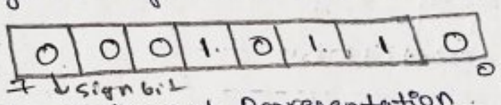


→ for +ve' numbers MSB bit is zero.
→ for -ve numbers MSB bit is one

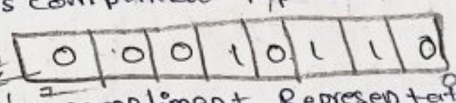
(1) Represent 22 in fixed point Representation

sol

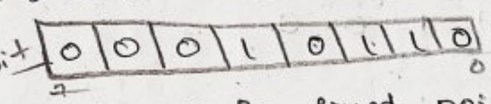
(1) Signed Magnitude Representation

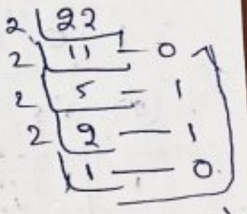


(2) 1's complement Representation



(3) 2's compliment Representation.

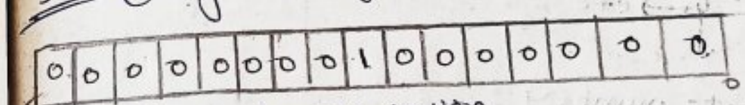




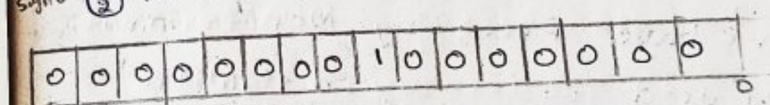
$(22)_{10} = (10110)_2$

(2) Represent 128 in fixed point Representation.

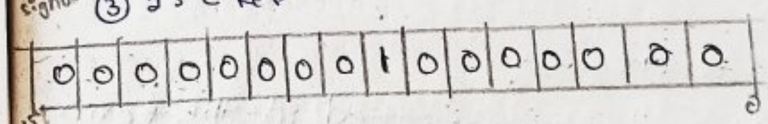
sol (1) signed Magnitude Representation



(2) i's C Representation



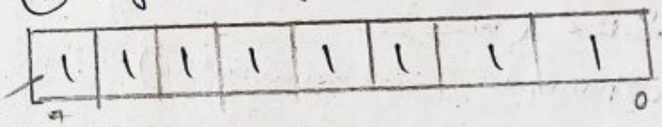
(3) 2's C Representation.



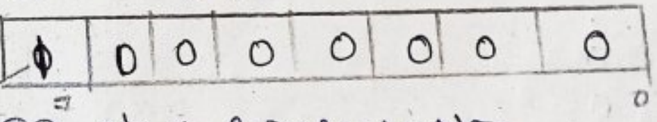


(3) Represent -127 in fixed point Represention.

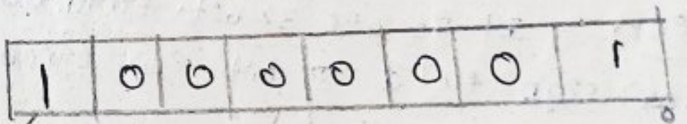
sol (1) Signed magnitude Representation



(2) 1's C Representation



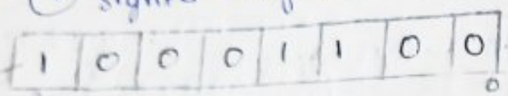
(3) 2's C Representation



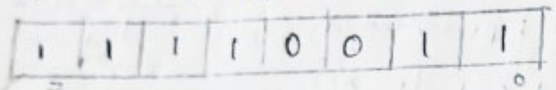
Scanned with OKEN Scanner

④ Represent -12 in (FPR)

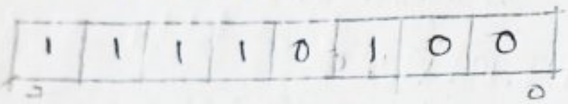
Sol ① Signed Magnitude



② 1's C Representation



③ 2's C Representation



Examples of fixed point numbers.

1. 10.4 , 1.05 } → etc - - -
2. 22.35

* floating point numbers :

These numbers have Recurring, Non-terminating fractional values used for accuracy in significe calculations.

Ex :- $10.\overline{3}$, π, etc - - -

floating point numbers are represented in the form

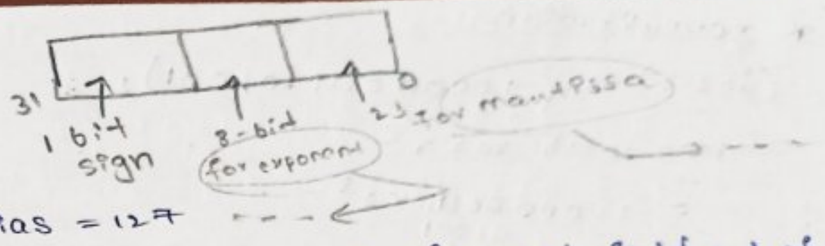
$m \times r^e$

where m = mantissa (integer value)
r = radix (Base)
e = exponent (fractional value)

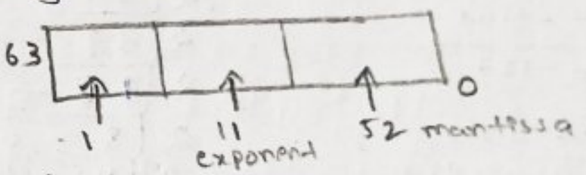
Types :-

1. Single precision.
2. Double precision.

1. single precision :- It is of 32 bit field out of which 1 bit for sign field 8 bit for exponent. 23 bit for mantissa.



2. Double precision :- It is of 64 bit field out of which 1 bit for sign field, 11 bits for exponent, remaining 52 bits for mantissa.



→ bias = 1023

Procedure for floating point :- (it is for both 32 & 64)

* Represent the number in binary.
* convert it into $m \times r^e$ form → this is called normalization.
* exponent value should be compared with 2^{e-bias}
* Represent m, S, e in frame format. (SEM)

① Represent 263.3 in 32 bit floating point representation (or) notation (or) single precision (or) IEEE 754 notation

Sol * 263.3
→ 263 + 0.3

2 | 263	
2 | 131	— 1
2 | 65	— 1
2 | 32	— 1
2 | 16	— 0
2 | 8	— 0
2 | 4	— 0
2 | 2	— 0
1	— 0

263 = $(100000111)_2$

0.3 × 2 = 0.6 = 0 ↓
0.6 × 2 = 1.2 = 1
0.2 × 2 = 0.4 = 0
0.4 × 2 = 0.8 = 0
0.8 × 2 = 1.6 = 1

$(263.3)_{10} = (100000111.01001)_2$

$(263.3)_{10} = (100000111.01001)_2$

Scanned with OKEN Scanner

* normalization

$(263.3) = (100000111.01001)_2$

compare with $m \times r^e$

$= 1.00000111\,01001 \times 2^8$

$m = 00000111\,01001$; $r = 2$, $e = 8$

* $2^8 = 2^{\epsilon\text{-bias}}$

$8 = \epsilon - \text{bias}$

$8 = \epsilon - 127$

$\epsilon = 135$

$135 = (10000111)_2$, $\epsilon = 10000111$

$S = 0$

2	135
2	67 — 1
2	33 — 1
2	16 — 1
2	8 — 0
2	4 — 0
2	2 — 0
	1 — 0

* frame formate

31	30 … 23	22 … 0
0	10000111	00001110101 … 00000
sign	exponent	Mantissa

→ 32 bit format ←

(2) Represent 12.5 in 32 bit floating pant representation.

Sol $12.5 = 12 + 0.5$

$(12)_{10} = (1100)_2$ $\quad 0.5 \times 2 = 1.0 = 0$

$12.5 = 1100.1$

* normalization.

$12.5 = 1100.1$

$= 1.1001 \times 2^3$ → comparing with $m \times r^e$

$m = 1001$; $r = 2$; $e = 3$

* $2^3 = 2^{\epsilon - \text{bias}}$

$\Rightarrow 3 = \epsilon - 127$

$\Rightarrow \epsilon = 130$

$\epsilon = 10000010$

$S = 0$

$M = 1001$

31	30 … 23	22 … 0
0	10000010	1001 … 00 … 00 … 0
Sign	Exponent	Mantissa

→ 32 bit format ←

2	130
2	65 — 0
2	32 — 1
2	16 — 0
2	8 — 0
2	4 — 0
2	2 — 0
	1 — 0

(3) Represent 1259.125 in 32 bit floating point representation.

Sol $1259.125 = 1259 + 0.125$

$0.125 \times 2 = 0.25 = 0$
$0.25 \times 2 = 0.5 = 0$
$0.5 \times 2 = 1.0 = 1$

$1259.125 = (10011101011.001)_2$

* normalization

$1259.125 = (10011101011.001)_2$

$\Rightarrow = 1.0011101011001 \times 2^{10}$

$m = 00111010011001$; $r = 2$; $e = 10$

* $2^{10} = 2^{\epsilon - 127}$

$\epsilon = 137$

$(137)_{10} = 10001001$

$\epsilon = 10001001$, $S = 0$, $M = 00111010011001$

2	1259
2	629 — 1
2	314 — 1
2	157 — 0
2	78 — 1
2	39 — 0
2	19 — 1
2	9 — 1
2	4 — 1
2	2 — 0
	1 — 0

* frame formate.

31	30 … 23	22 … 0
0	10001001	00111010011001 … 00 … 0
sign	exponent	Mantissa

→ 32 bit format ←

(4) Represent, 263.3 in 64 bit.

Sol $(263.3) = (100000111.01001)_2$

$= 1.0000011101001 \times 2^8$

$m = 0000011101001$; $r = 2$; $e = 8$

Scanned with OKEN Scanner

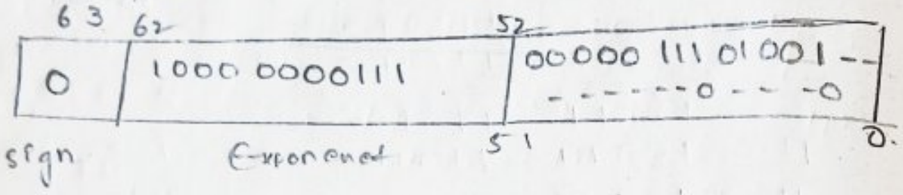
* $2^8 = 2^{\epsilon - bias}$

$8 = \epsilon - bias$

$\epsilon = 8 + 1023$

$\epsilon = 1031$ $\epsilon = 10000000111$

$(1031)_{10} = (10000000111)_2$



(5) Represend 12.5^{-} in 64-bit

Sol $12.5 = (1100.1)_2$

* normalization.

$12.5 = (1100.1)_2$

$\Rightarrow = 1.1001 \times 2^3$

$m = 1001$; $r = 2$; $e = 3$

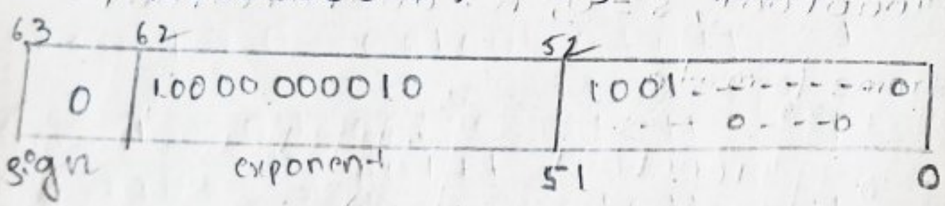
* $2^3 = 2^{\epsilon - bias}$

$3 = \epsilon - 1023.$

$\epsilon = 1026$.

$(1026)_{10} = (10000000010)$

$\epsilon = 10000000010$; $S = 0$; $M = 1001$



(6) Represent 1259.125 in 64 bit floating point Representation.

Sol $1259.125 = 1259 + 0.125$

$(1259.125)_{10} = (10011101011.001)_2$

* normalization :-

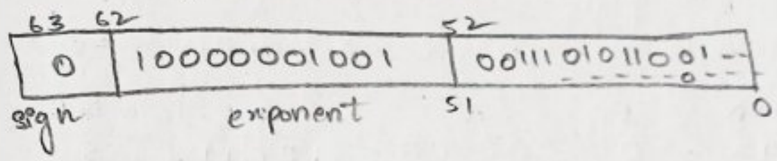
$= 1.0011101011001 \times 2^{10}$

$m = 0011101011001$; $e = 10$; $r = 2$

* $2^{10} = 2^{\epsilon - bias}$

$10 = \epsilon - 1023 \Rightarrow \epsilon = 1033$

$\epsilon = 10000001001$



*** Multiplication Algorithm:

→ used for +ve numbers only (unsigned)

→ if x, y are the I/p's & , x*y are o/p'

where, x = multiplicand = B.

y = multiplier

→ multiplicand is stored in B register

→ multiplier is stored in Q register

→ the operation should be performed upto sequence counter is 0. (upto SC=0)

→ Type of operation → SHR

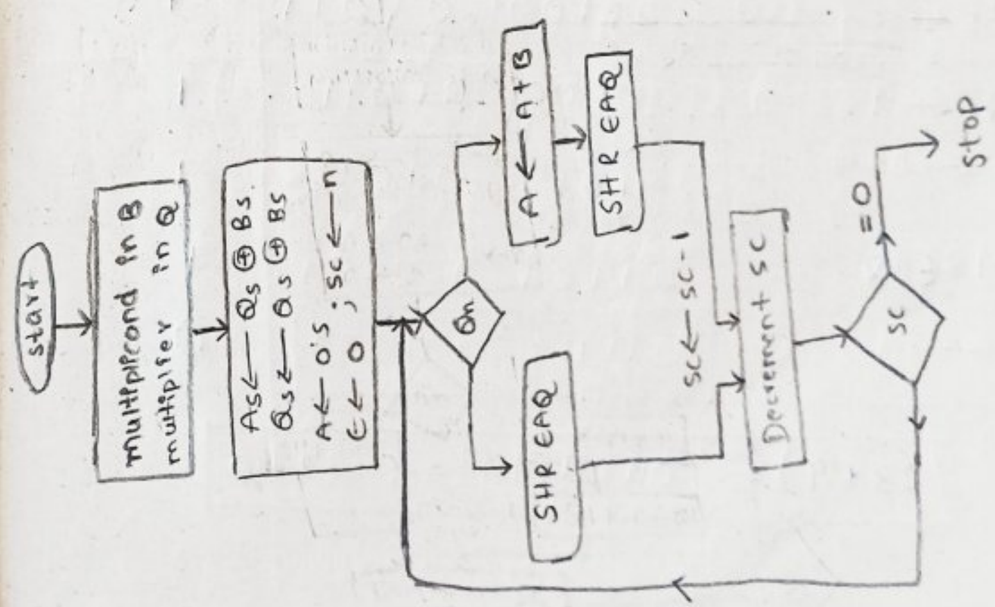
→ operation depends upon Q_n(LSB)

→ Here A, E are initially set to '0'

→ the final result is stored in A and Q in Binary.

→ when SC = 0

flow chart :-



Scanned with OKEN Scanner

① Perform 23 × 19 using multiplication algorithm

Sol Here, multiplicand (B) = 23 = $(10111)_2$
multiplier (Q) = 19 = $(10011)_2$

Sequence counter (S.C) = 5, A = 00000, E = 0

Operation	E	A	Q	S.C
$Q_n \leftarrow 1$	0	00000	10011	5
$A \leftarrow A+B$	0	10111 / 10111	10011	5
SHR EAQ		01011	11001	5
$SC \leftarrow SC-1$	0	01011	11001	4
$Q_n \leftarrow 1$	0	01011 / 10111	11001	4
$A \leftarrow A+B$	1	00010	11001	4
SHR EAQ	0	00001	01100	4
$SC \leftarrow SC-1$	0	10001	01100	3
$Q_n \leftarrow 0$	0	10001	01100	3
~~A ← A+B~~				
SHR EAQ		01000	10110	3
$SC \leftarrow SC-1$	0	01000	10110	2
$Q_n \leftarrow 0$	0	01000	10110	2
SHR EAQ		00100	01011	2
$SC \leftarrow SC-1$	0	00100	01011	1
$Q_n \leftarrow 1$	0	00100 / 10111	01011	1
$A \leftarrow A+B$	0	11011	01011	1
SHR EAQ		01101	10101	1
$SC \leftarrow SC-1$	0	01101	10101	0

$(23 \times 19) = (011011010 1 = (437)_{10}$

② Perform 12 × 11 using multiplication algorithm

Sol Here, multiplicand (B) = 12 = $(1100)_2$
multiplier (Q) = 11 = $(1011)_2$

SC = 4, A = 0000, E = 0

Operation	E	A	Q	S.C
$Q_n \leftarrow 1$	0	0000	1011	4
$A \leftarrow A+B$	0	1100 / 1100	1011	4
SHR EAQ		0110	0101	4
$SC \leftarrow SC-1$	0	0110	0101	3
$Q_n \leftarrow 1$	0	10110	0101	3
$A \leftarrow A+B$	1	1100 / 0010	0101	3
SHR EAQ		10010	00101	3
$SC \leftarrow SC-1$	0	10010	00101	2
$Q_n \leftarrow 0$	0	10010	00101	2
SHR EAQ		01001	10010	2
$SC \leftarrow SC-1$	0	01001	10010	1
$Q_n \leftarrow 1$	0	01001	10010	1
$A \leftarrow A+B$	1	1100 / 0101	10010	1
SHR EAQ		10100	01001	1
$SC \leftarrow SC-1$	0	10100	01001	0

$(12 \times 11) = (10000100)_2 = (132)_{10}$

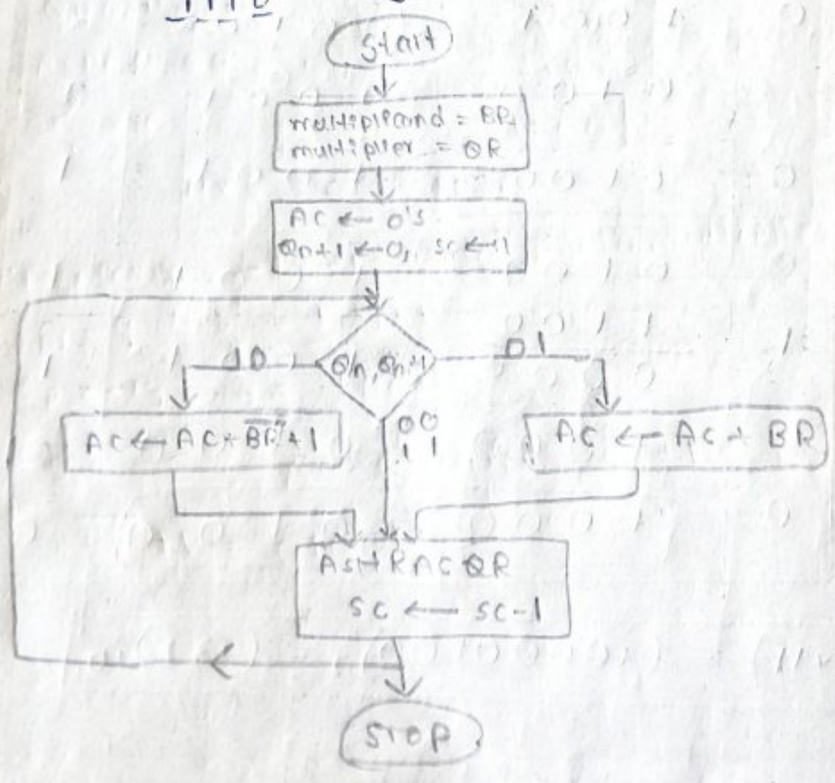
Scanned with OKEN Scanner

* Booth Multiplier :-

→ used for both 'tve' and 've' numbers.
→ Operation ASHR
→ If x & y are inputs, x * y is o/p → o/p is Direct form
→ If -x & y are inputs, -xy is o/p → o/p is 2's of Answer
→ If x & -y are inputs, -xy is o/p → o/p is 2's of Answer
→ If -x & -y are inputs, xy is o/p → o/p is Direct form

* Initially take A, C 0's (AC = 0's)
* $Q_{n+1} = 0$
* Operation depends upon Q_n, Q_{n+1} ← 00, 01, 10, 11
* Multiplicand in BR. (x = BR)
* multiplier in QR (y = QR)

* Ex :- 1100 x → Arithmatic shift Right
1110



① Perform -4 x 3 using Booth Multiplier.

Sol Here, BR = -4 → (1100) → 2's of 4 and for - place 1 to as MSB
QR = 3 → (0011) → as it is because of + place 0 to as MSB

SC = 4, AC = 0000, $\overline{BR}+1$ = 0100

$\overline{BR}+1$ = 1100 + 0011 + 1 = 0100

Operation	AC	QR	Qn+1	S.C
$Q_n Q_{n+1} \rightarrow 10$	0000	0011	0	4
AC ← AC + $\overline{BR}$+1	0100			
	0100	0011	0	4
AShR AC, QR	0010	0001	1	4
SC ← SC - 1	0010	0001	1	3
$Q_n Q_{n+1} \rightarrow 11$	0010	0001	1	3
AShR AC, QR	0001	0000	1	3
SC ← SC - 1	0001	0000	1	2
$Q_n Q_{n+1} \rightarrow 01$	0001	0000	1	2
	1100			
AC ← AC + BR	1101	0000	1	2
AShR AC QR	1110	1000	0	2
SC ← SC - 1	1110	1000	0	1
$Q_n Q_{n+1} \rightarrow 00$	1110	1000	0	1
AShR AC QR	1111	0100	0	1
SC ← SC - 1	1111	0100	0	0

AC & QR ⇒ 11110100

-4 x 3 = -12 = 11110100

-12 = 1100 + 0011 + 1 = 0100

② perform 4 x 3 using Booth multiplier.

Sol Here, BR = 4 → 0100
QR = 3 → 0011
$\overline{BR}$ = 1011

$\overline{BR}+1$ = 1011 + 1 = 1100

Scanned with OKEN Scanner

Operation	AC	QR	Q_{n+1}	SC
$Q_n Q_{n+1} \to 10$	0000	0011	0	4
$AC \leftarrow AC + \overline{BR} + 1$	1100 / 1100		0	4
ASHR AC, QR	1110	0001	1	4
$SC \to SC - 1$	1110	0001	1	3
$Q_n Q_{n+1} \to 11$	1110	0001	1	3
ASHR AC, QR	1111	0000	1	3
$SC \to SC - 1$	1111	0000	1	2
$Q_n Q_{n+1} \to 01$	1111	0000	1	2
$AC \leftarrow AC + BR$	0100 / 0011		1	2
ASHR AC, QR	0001	1000	0	2
$SC \to SC - 1$	0001	1000	0	1
$Q_n Q_{n+1} \to 00$	0001	1000	0	1
ASHR AC, QR	0000	1100	0	1
$SC \to SC - 1$	0000	1100	0	0

$11 \times 3 = 00001100$

③ Perform, -9×-13 using Booth multipliers

sol $BR = -9 \to = 10111$

$QR = -13 \to = 10011$

S.C = 5, AC = 00000, $Q_{n+1} = 0$, $\overline{BR} = 01000$

$\overline{BR} + 1 = 01001$

Operation	AC	QR	Q_{n+1}	SC
$Q_n Q_{n+1} \to 10$	00000	10011	0	5
$AC \leftarrow AC + \overline{BR} + 1$	01001 / 01001		0	5
ASHR AC, QR	00100	11001	1	5
$SC \to SC - 1$	00100	11001	1	4
$Q_n Q_{n+1} \to 11$	00100	11001	1	4
ASHR AC, QR	00010	01100	1	4
$SC \to SC - 1$	00010	01100	1	3
$Q_n Q_{n+1} \to 01$	00010	01100	1	3
$AC \leftarrow AC + BR$	10111 / 11001		1	3
ASHR AC, QR	11100	10110	0	3
$SC \to SC - 1$	11100	10110	0	2
$Q_n Q_{n+1} \to 00$	11100	10110	0	2
ASHR AC, QR	11110	01011	0	2
$SC \to SC - 1$	11110	01011	0	1
$Q_n Q_{n+1} \to 10$	11110	01011	0	1
$AC \leftarrow AC + \overline{BR} + 1$	01001 / 00111		0	1
ASHR AC, QR	00011	10101	1	1
$SC \to SC - 1$	00011	10101	1	0

$-9 \times -13 = 00011\,10101$

paper sum what are floating-point numbers -10.25

IEEE 754 single-precision format.

sol $-10.25 = 10 + 0.25$

$10 \to 1010$. $0.25 \times 2 = 0.5 = 0$ ↓

$0.5 \times 2 = 1 = 1$

$(10.25)_{10} = (1010.01)_2$.

normalisation, $= m \times r^e$.

$= 1010.01$

$= 1.01001 \times 2^3$

$e = 3, r = 2$

$m = 01001,$ ~~$e = 1000$~~

$2^3 = 2^{e\text{-bias}}$

$3 = e - 127$

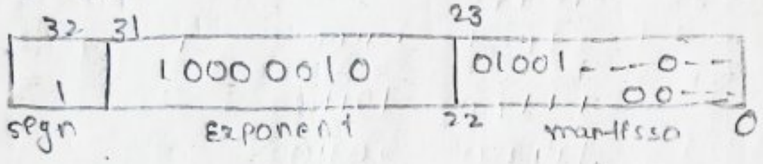
$e = 130$

$= 10000010$

$m = 01001$

$e = 1000\ 0010$

$s = 1$



64-bit

$2^3 = 2^{e\text{-bias}}$

$3 = e - 1023$

$e = 1023 + 3 = 1026$

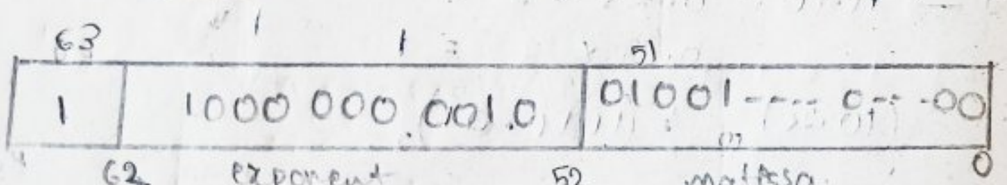
2 | 1026
2 | 513 — 0
2 | 256 — 1
2 | 128 — 0
2 | 64 — 0
2 | 32 — 0
2 | 16 — 0
2 | 8 — 0
2 | 4 — 0
2 | 2 — 0
1 — 0

$(1026)_{10} = (10000000010)_2$

$e = 10000000010$

$M = 01001$

$S = 1$



Sum: perform 6 x 5 using Multiplication algorithm

sol: Here, multiplicand (B) = 6 = (110)

multiplier (Q) = 5 = (101)

S.C = 3, A = 000, e = 0

operation	e	A	Q	S.C
$Q_n \leftarrow 1$	0	000	101	3
$A \leftarrow A+B$	0	110		3
		110	101 x	
SHR EAQ		011	010	3
$SC \leftarrow SC-1$	0	011	010	2
$Q_n \leftarrow 0$	0	011	010 x	2
SHR EAQ		001	101	2
$SC \leftarrow SC-1$	0	001	101	1
$Q_n \leftarrow 1$	0	001	101	1
$A \leftarrow A+B$	0	110		1
		111	101 x	
SHR EAQ		011	110	1
$SC \leftarrow SC-1$	0	011	110	0

$6 \times 5 = 30$

$6 \times 5 = 011110$

* Division algorithm :-

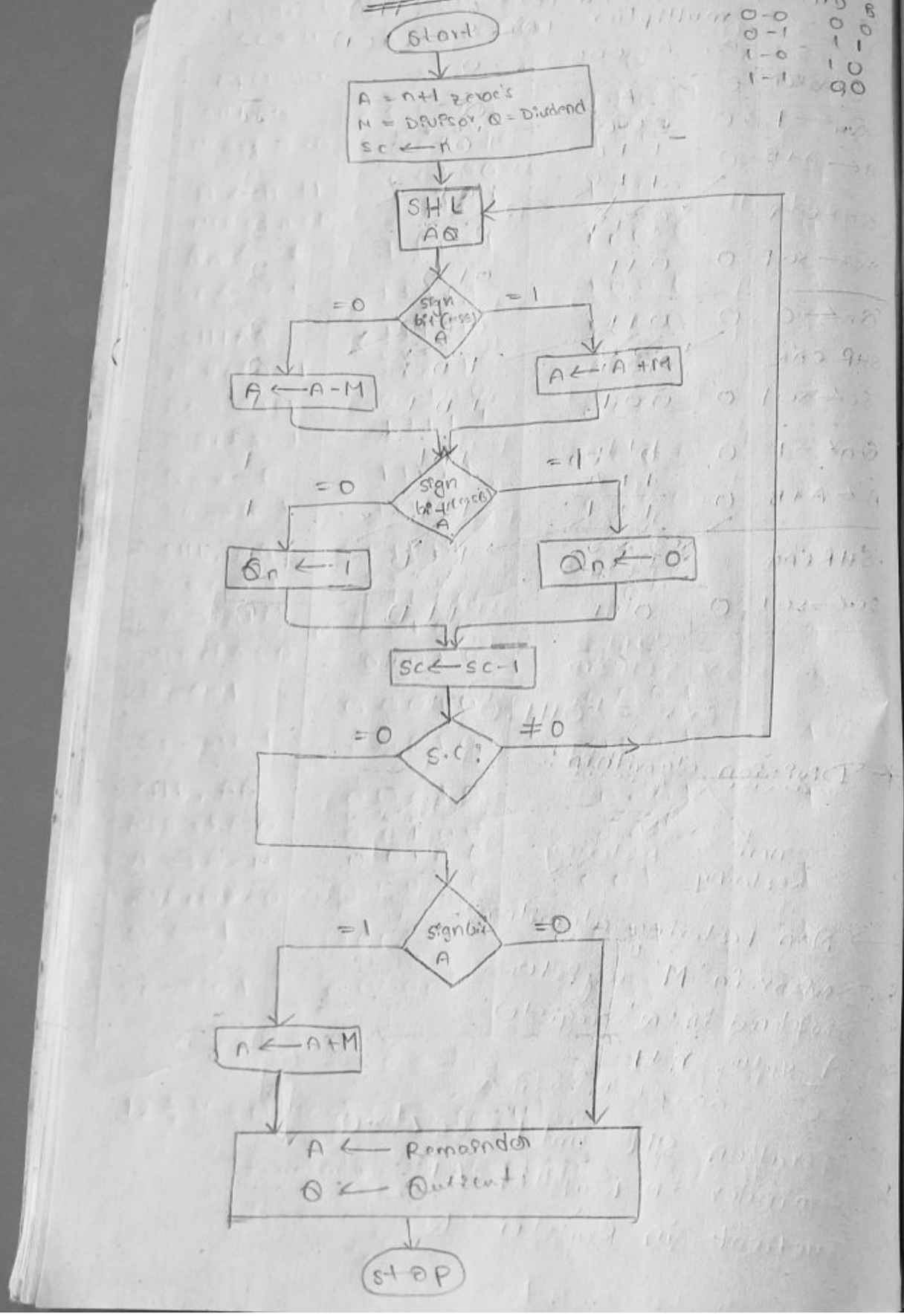
non Restoring | Restoring

→ Non Restoring algorithm :-

* Divisor in 'M' register
* Dividend in 'Q' register
* 'A' value n+1
* S.C → no. of bits in Q
* operation SHL until, SC ↔ 0
* Reminder in Register 'A'
* Quotient in Register 'Q'

Scanned with OKEN Scanner

flocochart



Subtr

0-0	0
0-1	1
1-0	1
1-1	0

(1) perform DVPSion operation for $11 \div 3$

Sol: Dividend = $11 = (1011)_2 = Q$; DVPSor = $3 = (00011)_2 = M$

S.C = 4 ; A = 00000

Operation	A	Q	S.C
SHLAQ	0 0 0 0 0	1 0 1 1	4
MSB of A = 0	0 0 0 0 1	0 1 1 _	
A ← A - M	1 1 1 1 0	0 1 1 _	4
MSB of A = 1			
Qn ← 0	1 1 1 1 0	0 1 1 0	4
Sc ← SC-1	1 1 1 1 0	0 1 1 0	3
SHL AQ	1 1 1 1 0	0 1 1 0	3
MSB of A = 1	1 1 1 0 0	1 1 0 _	
A ← A + M	0 0 0 1 1		
MSB of A = 1	1 1 1 1 1	1 1 0 _	3
Qn ← 0	1 1 1 1 1	1 1 0 0	3
Sc ← SC-1	1 1 1 1 1	1 1 0 0	2
SHL AQ	1 1 1 1 1	1 1 0 0	2
MSB of A = 1	1 1 1 1 1	1 0 0 _	
A ← A + M	0 0 0 1 1		
MSB of A = 0	0 0 0 1 0	1 0 0 _	2
Qn ← 1	0 0 0 1 0	1 0 0 1	2
Sc ← SC-1	0 0 0 1 0	1 0 0 1	1
SHL AQ	0 0 0 1 0	1 0 0 1	1
MSB of A = 0	0 0 1 0 1	0 0 1 _	
A ← A - M	0 0 0 1 1		
MSB of A = 0	0 0 0 1 0	0 0 1 _	1
Qn ← 1	0 0 0 1 0	0 0 1 1	1
Sc ← SC-1	0 0 0 1 0	0 0 1 1	0

Remainder — Quotient

$11 \div 3 \Longrightarrow$ Quotient $= (0011)_2 = (3)_{10}$

Remainder $= (00010)_2 = (2)_{10}$

Scanned with OKEN Scanner

(2) perform division operation for 12 ÷ 4.

Dividend = 12 = $(1100)_2$ = Q; Divisor = 4 = $(0100)_2$ = M

SC = 4 ; A = 00000

Operation	A	Q	S.C.
SHL AQ	00000	1100	4
MSB of A = 0	00001	100—	
A ← A-M	00100	100—	4
	11101		
MSB of A = 1	11101	100 0	4.
Q_n ← 0	11101		3
SC ← SC-1	11101	1000	
SHL AQ	11101	1000	3
MSB of A = 1	11011	000—	
A ← A+M	00100	000—	3.
	11111		
MSB of A = 1	11111	0000	3
Q_n ← 0	11111	0000	2
SC → SC-1	11111	0000	
SHL, AQ	11111	0000	2
MSB of A = 1	11110	000—	
A → A+M	00100		
	00010	000—	2.
MSB of A = 0	00010	0001	2
Q_n ← 1	00010	0001	1
SC → SC-1	00010	0001	
SHL, AQ	00010	0001	1
MSB of A = 0	00100	001—	
A → A-M	00100	001—	1
	00000		
MSB of A = 0	00000	0011	1
Q_n ← 1	00000	0011	0
SC → SC-1	00000	0011	

Remainder → 00000 ; Quotient → 0011

12 ÷ 4 = Quotient → $(0011)_2 = (3)_{10}$

Remainder → $(00000)_2 = (0)$